

## AIDS-I Assignment No: 2

### Q.1: Use the following data set for question 1

82, 66, 70, 59, 90, 78, 76, 95, 99, 84, 88, 76, 82, 81, 91, 64, 79, 76, 85, 90

1. Find the Mean (10pts)
2. Find the Median (10pts)
3. Find the Mode (10pts)
4. Find the Interquartile range (20pts)

Ans] **1. Mean** Mean=Sum of all values / Number of values=1611 / 20=80.55

### 2. Median

First, arrange the data in ascending order:

59, 64, 66, 70, 76, 76, 76, 78, 79, 81, 82, 82, 84, 85, 88, 90, 90, 91, 95, 99

Since there are 20 values (even), the median is the **average of the 10th and 11th values**:

$$\text{Median} = (81 + 82) / 2 = 163 / 2 = 81.5$$

### 3. Mode

Mode = value(s) that occur most frequently. Here, 76 appears 3 times, more than any other value.

$$\text{Mode} = 76$$

### 4. Interquartile Range (IQR) (20 pts):

Using the sorted data again:

59, 64, 66, 70, 76, 76, 76, 78, 79, 81, | 82, 82, 84, 85, 88, 90, 90, 91, 95, 99

- **Q1 (Lower Quartile)** = Median of the first 10 values = average of 5th and 6th values:

$$Q1 = (76 + 76) / 2 = 76$$

- **Q3 (Upper Quartile)** = Median of the last 10 values = average of 5th and 6th values (from 82–99):

$$Q3 = 85 + 88 / 2 = 86.5$$

$$\text{IQR} = Q3 - Q1 = 86.5 - 76 = 10.5$$

**Q.2 1) Machine Learning for Kids 2) Teachable Machine**

1. For each tool listed above
  - identify the target audience
  - discuss the use of this tool by the target audience
  - identify the tool's benefits and drawbacks
2. From the two choices listed below, how would you describe each tool listed above? Why did you choose the answer?
  - Predictive analytic
  - Descriptive analytic
3. From the three choices listed below, how would you describe each tool listed above? Why did you choose the answer?
  - Supervised learning
  - Unsupervised learning
  - Reinforcement learning

**Ans] 1) Machine Learning for Kids**

Machine Learning for Kids is a platform designed primarily for **school students and beginners** who are new to artificial intelligence and machine learning concepts. Its target audience includes **K-12 learners and educators** seeking to introduce AI through interactive, project-based learning. This tool allows students to build machine learning models using simple training datasets and integrate them into coding environments like Scratch or Python. It provides hands-on experience with text, image, and number-based classification models, fostering conceptual understanding in an intuitive manner.

The primary benefit of Machine Learning for Kids is its **accessibility and simplicity**—no prior knowledge of data science is required, making it ideal for educational environments. It supports **supervised learning**, as users label their own training data before training the model. A notable drawback is that its models are limited in complexity and not suitable for high-performance or production-level applications.

In terms of analytics type, this tool falls under **predictive analytics**, since the models it creates are designed to make predictions based on labeled data (e.g., predicting emotion from text or recognizing animals in images). It uses **supervised learning** because users provide input-output pairs (i.e., labeled data), allowing the model to learn mappings between features and target outcomes.

**2) Teachable Machine**

Teachable Machine by Google is another beginner-friendly tool aimed at a **wide range of users**, from students and educators to hobbyists and early-stage developers. It allows users to train classification models on images, sounds, or poses through a graphical interface without writing code. Users simply upload or record data, label it, and the system trains a model in real time. This tool is particularly effective for quick prototyping and demonstrating machine learning concepts interactively.

Teachable Machine's key benefits include its **speed, ease of use, and browser-based deployment**—trained models can be exported and embedded in websites or apps. However, it has some limitations in terms of **scalability, performance tuning, and model interpretability**, as it abstracts away algorithmic details for simplicity. Similar to Machine Learning for Kids, Teachable Machine is a **predictive analytics** tool because it uses historical labeled data to forecast outcomes (e.g., detecting whether a person is sitting or standing from webcam input).

The tool utilizes **supervised learning**, as it requires users to label input data and then trains the model accordingly. For example, a user can teach the system to differentiate between a cat and a dog by labeling image examples for each class. Since it relies on labeled input-output pairs, it does not employ unsupervised or reinforcement learning techniques.

In conclusion, both platforms emphasize accessibility and educational value, using **supervised learning and predictive analytics** to help users build practical intuition for AI systems. While they abstract complex machine learning processes for ease of use, they serve as powerful entry points for learners and rapid prototyping alike.

### Q.3 Data Visualization: Read the following two short articles:

- Read the article Kakande, Arthur. February 12. "What's in a chart? A Step-by-Step guide to Identifying Misinformation in Data Visualization." *Medium*
- Read the short web page Foley, Katherine Ellen. June 25, 2020. "How bad Covid-19 data visualizations mislead the public." *Quartz*
- Research a current event which highlights the results of misinformation based on data visualization. Explain how the data visualization method failed in presenting accurate information. Use newspaper articles, magazines, online news websites or any other legitimate and valid source to cite this example. Cite the news source that you found.

Ans] Misinformation through flawed data visualizations has significantly impacted public perception and policy decisions, particularly during the COVID-19 pandemic. A notable example involves the Georgia Department of Public Health's presentation of COVID-19 case data in May 2020. The department released a bar chart intended to show trends in confirmed cases across the state's five most affected counties. However, the x-axis of the chart was not arranged chronologically; instead, dates were ordered non-sequentially, and counties were inconsistently sorted. This arrangement misleadingly suggested a consistent decline in cases, potentially influencing public opinion and policy decisions regarding reopening and safety measures. [The misrepresentation was highlighted by multiple sources, including the Columbia Law School's Climate Law Blog, which emphasized how such visual distortions can mislead the public and policymakers.](#)

This case underscores the critical importance of adhering to best practices in data visualization. Axes should be clearly labeled and follow a logical sequence, data should be accurately represented without manipulation, and visualizations must provide context to avoid misinterpretation. Failure to meet these standards can lead to widespread misinformation, eroding public trust and hindering effective response efforts during crises.

<https://libguides.lib.cwu.edu/c.php?g=625394&p=4391900>

<https://www.brookings.edu/articles/how-disinformation-defined-the-2024-election-narrative/>

<https://dl.acm.org/doi/10.1145/3613904.3642448>

**Q. 4 Train Classification Model and visualize the prediction performance of trained model required information**

- Data File: Classification data.csv
- Class Label: Last Column
- Use any Machine Learning model ( SVM, Naïve Base Classifier )

**Requirements to satisfy**

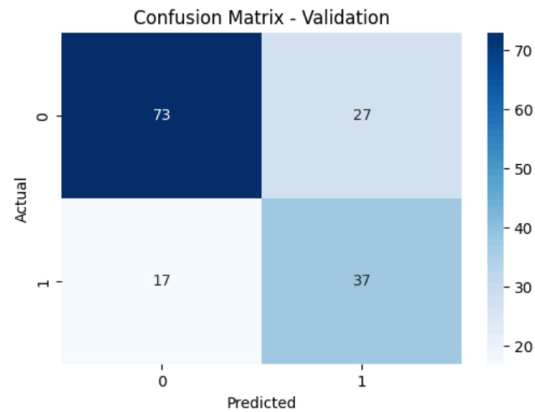
- Programming Language: Python
- Class imbalance should be resolved
- Data Pre-processing must be used
- Hyper parameter tuning must be used
- Train, Validation and Test Split should be 70/20/10
- Train and Test split must be randomly done
- Classification Accuracy should be maximized
- Use any Python library to present the accuracy measures of trained model

[Pima Indians Diabetes Database](#)

Ans] In this question, we developed a classification model to predict diabetes using the Pima Indians Diabetes dataset. The target variable was the 'Outcome' column, indicating the presence or absence of diabetes. We began by addressing class imbalance using the SMOTE (Synthetic Minority Over-sampling Technique), which balanced the dataset from 350:187 to 350:350 for each class, ensuring better learning for the minority class. We then performed data preprocessing, including feature scaling and appropriate train-validation-test splitting in a 70-20-10 ratio with random shuffling to avoid any bias. A Support Vector Machine (SVM) classifier was chosen, and hyperparameter tuning was conducted using GridSearchCV to optimize the model parameters. The best configuration was found with an RBF kernel and C=10. Upon evaluation, the model achieved a validation accuracy of 71% and a test accuracy of 79%. The test metrics showed a precision of 87% and recall of 80% for the non-diabetic class, and 68% precision with 78% recall for the diabetic class. Overall, the model performed well with balanced metrics across classes, especially after class imbalance was resolved, demonstrating its effectiveness in medical diagnosis scenarios.

Execution Link: [https://github.com/AdityaRaorane22/AIDS\\_CA2/blob/main/ca2a.ipynb](https://github.com/AdityaRaorane22/AIDS_CA2/blob/main/ca2a.ipynb)

|                     |           |        |          |         |
|---------------------|-----------|--------|----------|---------|
| Validation Metrics: |           |        |          |         |
|                     | precision | recall | f1-score | support |
| 0                   | 0.81      | 0.73   | 0.77     | 100     |
| 1                   | 0.58      | 0.69   | 0.63     | 54      |
| accuracy            |           |        | 0.71     | 154     |
| macro avg           | 0.69      | 0.71   | 0.70     | 154     |
| weighted avg        | 0.73      | 0.71   | 0.72     | 154     |
| Test Metrics:       |           |        |          |         |
|                     | precision | recall | f1-score | support |
| 0                   | 0.87      | 0.80   | 0.83     | 50      |
| 1                   | 0.68      | 0.78   | 0.72     | 27      |
| accuracy            |           |        | 0.79     | 77      |
| macro avg           | 0.77      | 0.79   | 0.78     | 77      |
| weighted avg        | 0.80      | 0.79   | 0.80     | 77      |



### Q.5 Train Regression Model and visualize the prediction performance of trained model

- Data File: Regression data.csv
- Independent Variable: 1st Column
- Dependent variables: Column 2 to 5

Use any Regression model to predict the values of all Dependent variables using values of 1st column.

#### Requirements to satisfy:

- Programming Language: Python
- OOP approach must be followed
- Hyper parameter tuning must be used
- Train and Test Split should be 70/30
- Train and Test split must be randomly done
- Adjusted R2 score should more than 0.99
- Use any Python library to present the accuracy measures of trained model

<https://github.com/Sutanoy/Public-Regression-Datasets>

<https://raw.githubusercontent.com/selva86/datasets/master/BostonHousing.csv>

[V](#)

- URL:

<https://archive.ics.uci.edu/ml/machine-learning-databases/00477/Real%20estate%20valuation%20data%20set.xlsx>

( Refer any one )

Ans] In this project, we aimed to build a **multi-output regression model** using the **Boston Housing dataset**, with the goal of predicting four target variables (**zn**, **indus**, **chas**, and **nox**) using only the **first feature crim** (per capita crime rate). We approached the problem using an **Object-Oriented Programming (OOP)** paradigm in Python, where a custom **MultiOutputRegressor** class was designed to handle training, evaluation, and visualization.

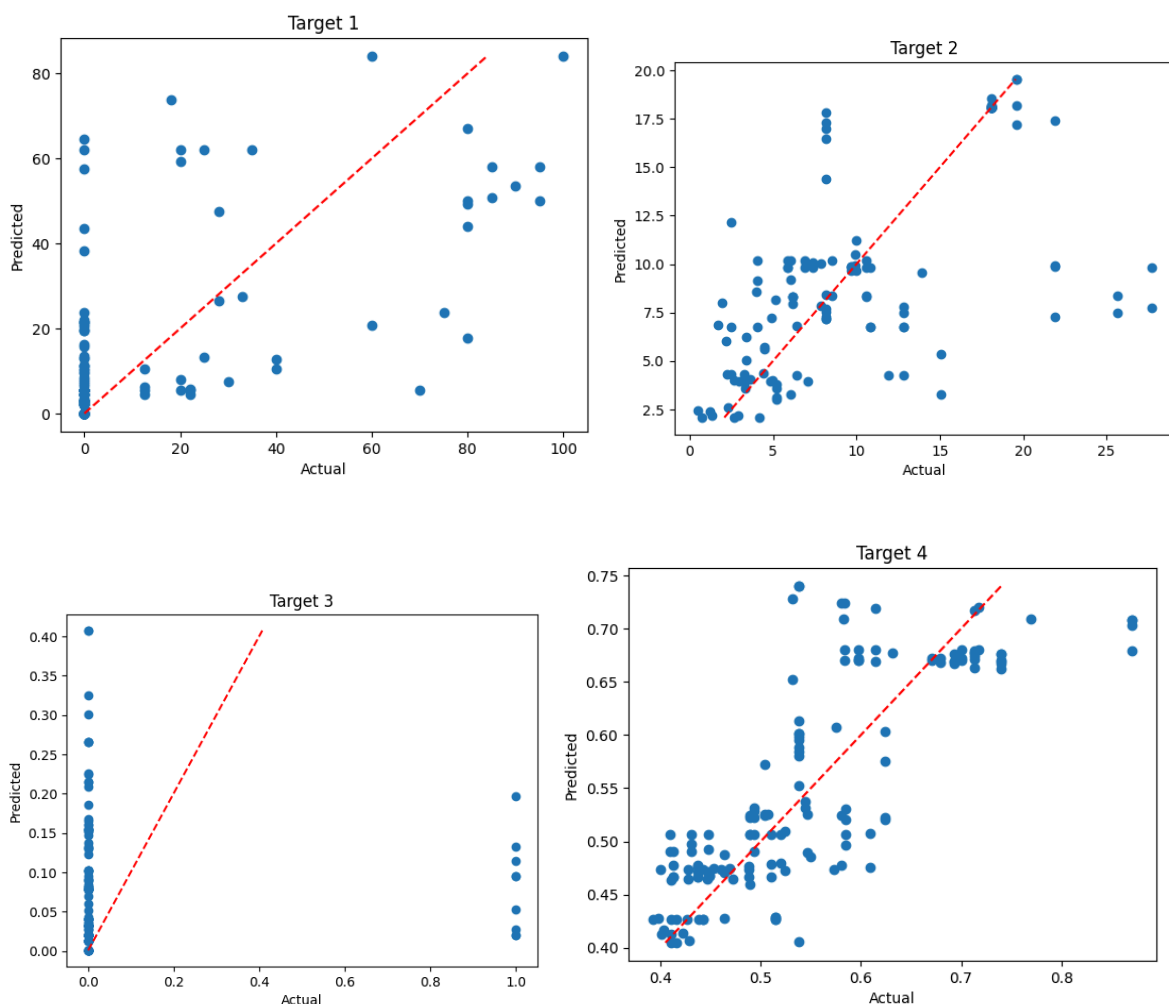
We split the dataset using a **70/30 train-test ratio**, and trained **individual RandomForestRegressor** models for each target variable. We also incorporated **hyperparameter tuning using GridSearchCV** to optimize model performance.

Upon evaluation using **Adjusted R<sup>2</sup> score**, we observed the following results:

- Output 1 (**zn**): 0.3880
- Output 2 (**indus**): 0.5035
- Output 3 (**chas**): -0.0959
- Output 4 (**nox**): 0.6406

These scores suggest that **crim** alone is **not strongly predictive** of these four housing-related attributes. The **low adjusted R<sup>2</sup> values**, particularly the **negative score for chas**, indicate that a single feature (**crim**) does not capture enough variance to make highly accurate predictions across multiple outputs. To improve performance, incorporating more features from the dataset would likely be necessary.

Execution Link: [https://github.com/AdityaRaorane22/AIDS\\_CA2/blob/main/ca2b.ipynb](https://github.com/AdityaRaorane22/AIDS_CA2/blob/main/ca2b.ipynb)



**Q.6** What are the key features of the wine quality data set? Discuss the importance of each feature in predicting the quality of wine? How did you handle missing data in the wine quality data set during the feature engineering process? Discuss the advantages and disadvantages of different imputation techniques. (Refer dataset from Kaggle).

Ans] The Wine Quality dataset from Kaggle contains 12 attributes, where 11 are input features based on physicochemical tests, and one is an output variable representing wine quality, scored between 0 and 10. The key features include: **fixed acidity**, which affects the taste and stability of wine; **volatile acidity**, which, in excess, leads to unpleasant vinegar-like flavors; **citric acid**, contributing to freshness and flavor; **residual sugar**, important for sweetness and mouthfeel; **chlorides**, indicating salt content that can affect taste; **free sulfur dioxide** and **total sulfur dioxide**, which act as preservatives to prevent oxidation and microbial growth; **density**, closely related to sugar and alcohol content; **pH**, indicating acidity levels; **sulphates**, which can enhance flavor and preservation; and **alcohol**, which often positively correlates with perceived quality. Each of these features plays a role in determining the overall sensory impression of the wine. During feature engineering, handling missing data is crucial for maintaining data integrity and model performance. If missing values were found in the dataset, techniques such as **mean or median imputation** (replacing missing values with the average or middle value) are simple and quick, but can lead to biased results if the data is not normally distributed. **Mode imputation** is suited for categorical data, though not applicable here. **K-Nearest Neighbors (KNN) imputation** offers more accurate filling based on similarity between data points but is computationally expensive. **Multivariate imputation** considers relationships among multiple features, offering better estimates but increasing model complexity. Each method involves a trade-off between accuracy and computational efficiency, and the choice depends on the data distribution and the intended use of the model.